

On This Page >

Inventory Submodule

This provides an overview of the available options of the `inventory` model.

It can be set via the `inventory` attribute of the `clan` function, or via the `clan.inventory` attribute of flake-parts.

instances

Attribute: `inventory.instances`

Multi host service module instances

Type: `attribute set of (submodule)`

Default:

```
{ }
```

Declared in: [modules/inventoryClass/inventory.nix](#)

module

Attribute: `inventory.instances.<name>.module`

This option has no description.

Type: `submodule`

Default:

```
{ }
```

Declared in: [modules/inventoryClass/inventory.nix](#)

input

Attribute: `inventory.instances.<name>.module.input`

Name of the input. Default to 'null' which means the module is local

Type: `null or string`

Default:

"Name of the input. Default to 'null' which means the module is local"

Declared in: [modules/inventoryClass/inventory.nix](#)

name

Attribute: `inventory.instances.<name>.module.name`

Attribute of the clan service module imported from the chosen input.

Defaults to the name of the instance.

Type: `string`

Default:

"<Name of the Instance>"

Declared in: [modules/inventoryClass/inventory.nix](#)

roles

Attribute: `inventory.instances.<name>.roles`

This option has no description.

Type: attribute set of (submodule)

Default:

{ }

Declared in: [modules/inventoryClass/inventory.nix](#)

extraModules

Attribute: inventory.instances.<name>.roles.<name>.extraModules

List of additionally imported `.nix` expressions.

Note

The import only happens if the machine is part of the service or role.

Other types are passed through to the nixos configuration.

Example

To import the `special.nix` file

```
1 . Clan Directory
2 |─ flake.nix
3 |
4 |─ modules
5 |   └─ special.nix
6 |   └─ ...
```

```
{
  extraModules = [ "modules/special.nix" ];
}
```

Type: list of raw value

Default:

[]

Declared in: [modules/inventoryClass/inventory.nix](#)

machines

Attribute: `inventory.instances.<name>.roles.<name>.machines`

This option has no description.

Type: `attribute set of (submodule)`

Default:

{ }

Declared in: [modules/inventoryClass/inventory.nix](#)

settings

Attribute: `inventory.instances.<name>.roles.<name>.machines.<name>.settings`

This option has no description.

Type: `deferred custom module. Must be JSON serializable.`

Default:

{ }

Declared in: [modules/inventoryClass/inventory.nix](#)

settings

Attribute: `inventory.instances.<name>.roles.<name>.settings`

This option has no description.

Type: deferred custom module. Must be JSON serializable.

Default:

```
{ }
```

Declared in: [modules/inventoryClass/inventory.nix](#)

tags

Attribute: inventory.instances.<name>.roles.<name>.tags

This option has no description.

Type: (attribute set of (submodule)) or (list of string) convertible to it

Default:

```
{ }
```

Declared in: [modules/inventoryClass/inventory.nix](#)

machines

Attribute: inventory.machines

Machines in the inventory.

Each machine declared here can be referenced via its `attributeName` by the `inventory.service`s `roles`.

Type: lazy attribute set of (submodule)

Default:

```
{ }
```

Declared in: [modules/inventoryClass/inventory.nix](#)

deploy

Attribute: `inventory.machines.<name>.deploy`

buildHost

Attribute: `inventory.machines.<name>.deploy.buildHost`

SSH address of the host to build the machine on

Type: `null or string`

Default:

`null`

Declared in: [modules/inventoryClass/inventory.nix](#)

targetHost

Attribute: `inventory.machines.<name>.deploy.targetHost`

SSH address of the host to deploy the machine to

Type: `null or string`

Default:

`null`

Declared in: [modules/inventoryClass/inventory.nix](#)

description

Attribute: `inventory.machines.<name>.description`

Optional freeform description

Type: `null or string`

Default:

`null`

Declared in: [modules/inventoryClass/inventory.nix](#)

icon

Attribute: `inventory.machines.<name>.icon`

Under construction, will be used for the UI

Type: `null or string`

Default:

`null`

Declared in: [modules/inventoryClass/inventory.nix](#)

installedAt

Attribute: `inventory.machines.<name>.installedAt`

Indicates when the machine was first installed.

Timestamp is in unix time (seconds since epoch).

Type: `null or signed integer`

Default:

`null`

Declared in: [modules/inventoryClass/inventory.nix](#)

machineClass

Attribute: `inventory.machines.<name>.machineClass`

The module system that should be used to construct the machine

Set this to `darwin` for macOS machines

Type: `one of "nixos", "darwin"`

Default:

`"nixos"`

Declared in: [modules/inventoryClass/inventory.nix](#)

name

Attribute: `inventory.machines.<name>.name`

Name of the machine or service

Type: `string`

Default:

`"<name>"`

Declared in: [modules/inventoryClass/inventory.nix](#)

tags

Attribute: `inventory.machines.<name>.tags`

List of tags for the machine.

The machine can be referenced by its tags in `inventory.services`

❖ Example

```
inventory.machines.machineA.tags = [ "tag1" "tag2" ];

services.borgbackup."instance_1".roles.client.tags = [ "tag1" ];
```

📘 Note

Tags can be used to determine the membership of the machine in the services. Without changing the service configuration, the machine can be added to a service by adding the correct tags to the machine.

Type: `list of string`

Default:

`[ ]`

Declared in: [modules/inventoryClass/inventory.nix](#)

meta

Attribute: `inventory.meta`

This option has no description.

Type: `submodule`

Declared in: [modules/inventoryClass/inventory.nix](#)

description

Attribute: `inventory.meta.description`

Optional freeform description

Type: `null or string`

Default:

`null`

Declared in: [modules/inventoryClass/meta.nix](#)

domain

Attribute: `inventory.meta.domain`

Domain for the clan.

It will be used to wire clan-internal services and resolve the address for each machine of the clan using `<hostname>.<meta.domain>`

This can either be:

- A top level domain (TLD). Set this to a valid, but not already existing TLD if you're using a mesh network between your machines. This will route requests between your machines over the mesh network.
- A regular domain. Set this to a valid domain you own if you want to route requests between your machines over the public internet. You will have to manually setup your public DNS of that domain to route `<hostname>.<meta.domain>` to each of your machines.

Type: `string matching the pattern  $^{[a-z0-9]}([a-z0-9-]{0,61}[a-z0-9])?(\.[a-z0-9]([a-z0-9-]{0,61}[a-z0-9])?)^*\$$`

Default:

`"clan"`

Example

`"clan.lol"`

Declared in: [modules/inventoryClass/meta.nix](#)

icon

Attribute: `inventory.meta.icon`

Under construction, will be used for the UI

Type: `null or string`

Default:

`null`

Declared in: [modules/inventoryClass/meta.nix](#)

name

Attribute: `inventory.meta.name`

Name of the clan.

Needs to be (globally) unique, as this determines the folder name where the flake gets downloaded to.

Should only contain alphanumeric characters, `_` and `-`.

Type: `string matching the pattern [a-zA-Z0-9_-]*`

❖ Example

```
"my_clan"
```

Declared in: [modules/inventoryClass/meta.nix](#)

tld

Attribute: `inventory.meta.tld`

Deprecated: Use `domain` instead.

Type: `null` or string matching the pattern `[a-z]+`

Default:

`null`

❖ Example

`"ccc"`

Declared in: [modules/inventoryClass/meta.nix](#)

modules

Attribute: `inventory.modules`

A mapping of module names to their path.

Each module can be referenced by its `attributeName` in the `inventory.services` attribute set.

💡 Important

Each module MUST fulfill the following requirements to be usable with the inventory:

- The module MUST have a `README.md` file with a `description`.
- The module MUST have at least `features = [ "inventory" ]` in the frontmatter section.
- The module MUST have a subfolder `roles` with at least one `{roleName}.nix` file.

For further information see: [Module Authoring Guide](#).

❖ Example

```
1 clan-core.lib.clan {
2     # 1. Add the module to the available inventory modules
3     inventory.modules = {
4         custom-module = ./modules/my_module;
```

```
5     };  
6     # 2. Use the module in the inventory  
7     inventory.services = {  
8         custom-module.instance_1 = {  
9             roles.default.machines = [ "machineA" ];  
10        };  
11    };  
12 };
```

Type: `attribute set of raw value`

Default:

`"clanModules of clan-core"`

Declared in: [modules/inventoryClass/inventory.nix](#)

tags

Attribute: `inventory.tags`

Tags of the inventory are used to group machines together.

It is recommended to use `machine.tags` to define the tags of the machines.

This can be used to define custom tags that are either statically set or dynamically computed.

Static Tags

❖ Static Tag Example

```
inventory.tags = {  
    foo = [ "machineA" "machineB" ];  
};
```

The tag `foo` will always be added to `machineA` and `machineB`.

Dynamic Tags

It is possible to compute tags based on the machines properties or based on other tags.

⚠ Danger

This is a powerful feature and should be used with caution.

It is possible to cause infinite recursion by computing tags based on the machines properties or based on other tags.

💡 Dynamic Tag Example

allButFoo is a computed tag. It will be added to all machines except 'foo'

`all` is a predefined tag. See the docs of [tags.all](#).

```
1 # inventory.tags ↓          ↓ inventory.machines
2 inventory.tags = {config, machines...}: {
3   #
4   allButFoo = builtins.filter (name: name != "foo") config.all;
5 };
```

↓↓↓ The "a"

⚠ Warning

Do NOT compute `tags` from `machine.tags` this will cause infinite recursion.

Type: `open submodule of lazy attribute set of list of string`

Default:

```
{ }
```

Declared in: [modules/inventoryClass/inventory.nix](#)

all

Attribute: `inventory.tags.all`

❖ Predefined Tag

Will be added to all machines

```
inventory.machines.machineA.tags = [ "all" ];
```

Type: `list of string`

Default:

```
"[ <All Machines> ]"
```

Declared in: [modules/inventoryClass/inventory.nix](#)

darwin

Attribute: `inventory.tags.darwin`

❖ Predefined Tag

Will be added to all machines that set `machineClass = "darwin"`

```
inventory.machines.machineA.tags = [ "darwin" ];
```

Type: `list of string`

Default:

```
"[ <All Darwin Machines> ]"
```

Declared in: [modules/inventoryClass/inventory.nix](#)

nixos

Attribute: `inventory.tags.nixos` **Predefined Tag**

Will be added to all machines that set `machineClass = "nixos"`

```
inventory.machines.machineA.tags = [ "nixos" ];
```

Type: `list of string`**Default:**

```
"[ <All NixOS Machines> ]"
```

Declared in: [modules/inventoryClass/inventory.nix](#)

[← Previous](#)[Clan Options](#)[Next →](#)
[Clan service](#)

Need help? Reach out to [our community](#).